Operator Overloading

1.Define operator overloading. (AU22)

Ans: Operator overloading is a feature in programming languages that allows the same operator to have different behaviors depending on the operands used.

2.Why is it necessary to overload an operator? (AU22)

Ans:

a. To make code more readable and concise.

b. To provide additional functionality to user-defined types.

c. To make code more efficient.

d. To make code more consistent.

3.Write the rules of operator overloading? (SP22)

Ans:

Operator Availability: Not all operators can be overloaded in every programming language. Each language defines which operators can be overloaded and the specific rules for each. **Number of Operands:** Most unary and binary operators can be overloaded. Unary operators like increment (++), decrement (--), and logical negation (!) can be overloaded with only one operand. Binary operators like addition (+), subtraction (-), and equality (==) can be overloaded with two operands. **Operand Types:** Operator overloading can be performed with user-defined types (classes) or built-in types (integers, strings, etc.). However, some languages have restrictions on which operators can be overloaded for built-in types.

4.What is the difference between operator function and normal function? Explain with a program example? (AU22)

Ans:

The main difference between operator functions and normal functions is that operator functions are specifically designed to work with operators, allowing userdefined objects to behave like built-in types and enabling the use of operators with those objects. On the other hand, normal functions are general-purpose functions that perform specific tasks or operations but are not directly associated with any particular operator or class.

Example:

```
#include <iostream>
using namespace std;
class A
{
private:
int a;
public:
A(int a) : a(a) {}
// Operator function
A operator+(const A& b){
return A(a + b.a);
}
// Normal function
int getValue()
{
return a;
}
};
int main()
{
A num1(1);
A num2(10);
A sum = num1 + num2;
cout << "Sum: " << sum.getValue() <<endl;
return 0;
}
```

5. Explain why a friend function can not be used to overload the assignment operator (=)?

Ans:

n C++, the assignment operator (=) cannot be overloaded using a friend function. The assignment operator is a special operator that is used to assign the value of one object to another object of the same class. It is typically implemented as a member function of the class.

Here are a few reasons why the assignment operator cannot be overloaded using a friend function:

Access to Private Members: The assignment operator is responsible for copying the values of all the data members from one object to another. To perform this operation, it needs direct access to the private members of both the source and destination objects. Friend functions, however, have access to the private members of only one class, not multiple classes simultaneously. Therefore, a friend function cannot access the private members of both the source and destination objects required for proper assignment.

Implicit Object Assignment: The assignment operator is implicitly called when you assign one object to another using the = operator. For example, obj1 = obj2; calls the assignment operator. Friend functions are not invoked implicitly in this manner. They require explicit function calls, which means that using a friend function to overload the assignment operator would break the typical syntax and behavior of object assignment

6.When operator overloading does it lose any of it's original functionality?

Ans:

When an operator is overloaded, it is possible for it to lose some or all of its original functionality, depending on how the operator is defined in the overloaded implementation. By overloading an operator, you provide a new definition for that operator when applied to specific types or classes. This allows you to customize the behavior of the operator to fit the requirements of your program or data types. However, it's important to note that the original functionality of an operator is not inherently lost when it is overloaded. The original functionality of an operator is preserved for the types or classes for which it was originally defined. When the operator is used with those types or classes, the original behavior will still apply.

7.Can the precedence of an overloaded operator be changed ? Can the number of operand be altered?

Ans:

In most programming languages, including C++, the precedence of an overloaded operator cannot be changed. The precedence of operators is determined by the language's syntax and grammar rules, and it is not affected by operator overloading.When you overload an operator, you define its behavior for specific types or classes,but you cannot modify its precedence. The precedence of the operator remains the same as defined by the language.

No, the number of operands for an operator cannot be altered when it is overloaded. The number of operands that an operator expects is a fundamental characteristic of the operator defined by the language's syntax and grammar rules. When you overload an operator, you provide a new definition for that operator when applied to specific types or classes. However, the arity (number of operands) of the operator remains unchanged

8.Difference between the member and friend function ?

Ans:

Friend Function	**Member function**
It can be declared in any number of classes using the keyword friend.	It can be declared only in the private, public, or protected scope of a particular class.
This function has access to all private and protected members of classes.	This function has access to private and protected members of the same class.
One can call the friend function in the main function without any need to object.	One has to create an object of the same class to call the member function of the class.
The Friend keyword is generally used to declare a function as a friend function.	In these, there is no such keyword required.

Points to Remember While Overloading Operator using Friend Function:

1. The Friend function in C++ using operator overloading offers better flexibility to the class.
2. The Friend functions are not a member of the class and hence they do not have ‘this’ pointer.
3. When we overload a unary operator, we need to pass one argument.
4. When we overload a binary operator, we need to pass two arguments.
5. The friend function in C++ can access the private data members of a class directly.
6. An overloaded operator friend could be declared in either the private or public section of a class.
7. You cannot use a friend to overload the assignment operator. The assignment operator can be overloaded only by a member operator function.

Inheritance

1. **"A derived class can access all the members of its base class."-Is this statement true?**
   Ans:
   Yes, the statement is true. In C++, a derived class can access all the members of its base class, including public, protected, and private members.
   Here's a brief explanation of member accessibility in C++ inheritance:
   a. Public members: Public members of the base class are accessible to the derived class.
   b. Protected members: Protected members of the base class are also accessible to the derived class. They can be accessed in the same way as public members.
   c. Private members: Private members of the base class are not directly accessible to the derived class. However, they are still present in the derived class and can be accessed through public or protected member functions of the base class.
2. **In inheritance, explain why the protected category is needed?**
   Ans:
   The protected access specifier in inheritance serves as an intermediate level of access control between public and private. The protected category is needed in inheritance for several reasons:
   Encapsulation: Protected members allow for encapsulation within the inheritance hierarchy. They provide a level of data and functionality that is not directly accessible to external classes but can be accessed by derived classes. This promotes the principle of information hiding and helps maintain the integrity of the baseclass.
   Frameworks and libraries: In larger projects, frameworks, and libraries, protected members can bused to expose certain functionality to derived classes while still hiding implementation details from the outside world. This helps in maintaining a well-defined API for the framework or library, allowing for easier extension and customization.
   Polymorphism: The protected access specifier is also essential for achieving polymorphism, which is a fundamental concept in object-oriented programming. Without protected members, derived classes wouldn't have access to the base class's protected members, limiting their ability to participate fully in polymorphic behavior
3. **How does the properties of the following two derived classes differ?**
   Class D1:private B{….}
   Class D1:public B{….}
   Ans:
   There are two derived classes, D1 and D2 both of which inherit from a base class B. However, the key difference between them is the access specifier used in the inheritance: class D1: private B
   In this case, D1 is inheriting from B as a private base class.
   All public and protected members of B will become private members of D1. This means that these members will not be accessible outside of the class D1.The base class B itself is not accessible from outside of D1.This is known as "private inheritance, " class D2: public B: In this case, D1 inherits from B using the public access specifier. This means that the public and protected members of B remain public and protected members, respectively, in D1.class D2: public B: In this case, D2 is inheriting from B as a public base class. All public and protected members of B will retain their access specifiers in D2. Public members will remain public, and protected members will remain protected. The base class B itself is also accessible from outside of D2. This is a traditional "public inheritance
4. **What the problem associate with multiple inheritance and how is it solve by virtual inheritance?**
   Ans:
   Multiple inheritance is a feature of some object-oriented programming languages that allows a class to inherit from multiple base classes. This can be useful for creating classes that have a complex set of behaviors or properties. However, multiple inheritance can also lead to problems, such as the diamond problem.

```
class A {
public:
int a;
};
class B : public A {
public:
int b;
};
class C : public A {
public:
int c;
};
class D : public B, public C {
};
```

In this code, the D class inherits from both the B class and the C class, which both inherit from the A class. This means that the D class has two copies of the A class's a member. This can lead to confusion when trying to access the a member. Virtual inheritance is a feature of some object-oriented programming languages that can be used to solve the diamond problem. Virtual inheritance allows a class to inherit from a base class only once, even if the base class is inherited from by multiple other classes. For example, the following code shows how the D class can be declared using virtual

```
inheritance:class A {
public:
int a;
};
class B : public virtual A {
public:
int b;
};
class C : public virtual A {
public:
int c;
};
class D : public B, public C {
};
```

In this code, the D class inherits from the B class and the C class, but it only inherits from the A class once. This means that the D class only has one copy of the A class's a member.

Modes of Inheritance:

Public Mode: If we derive a subclass from a public base class. Then the public member of the base class will become public in the derived class and protected members of the base class will become protected in the derived class.

Protected Mode: If we derive a subclass from a Protected base class. Then both public members and protected members of the base class will become protected in the derived class.

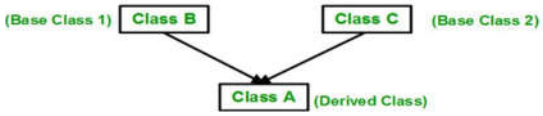
Private Mode: If we derive a subclass from a Private base class. Then both public members and protected members of the base class will become Private in the derived class. The below table summarizes the above three modes and shows the access specifier of the members of the base class in the subclass when derived in public, protected and private modes:

Base class member access specifier	Type of Inheritence		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

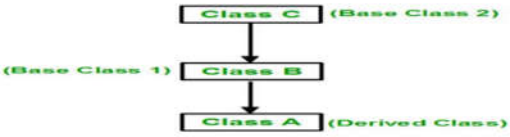
Types of Inheritance in C++

Single Inheritance: In Single Inheritance, one class is derived from another class.
• It represents a form of inheritance where there is only one base and derived class.

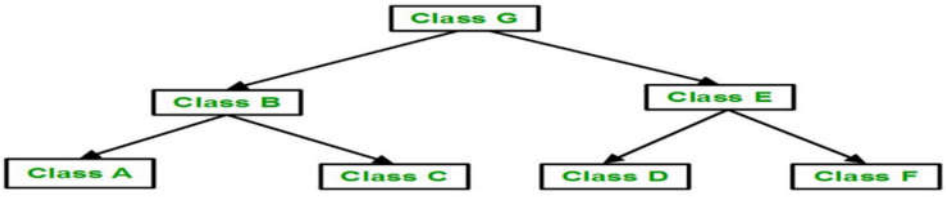
Multiple Inheritance: Multiple Inheritance is a feature of C++ where a class can inherit from more than one class. i.e one **subclass** is inherited from more than one **base class**



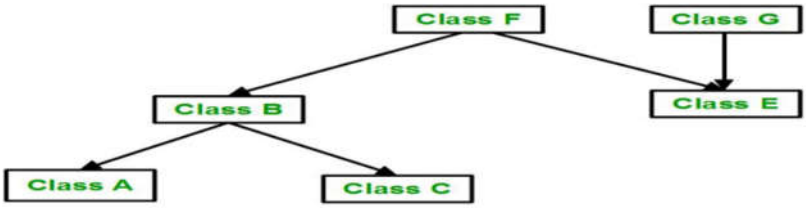
Multilevel Inheritance: In this type of inheritance, a derived class is created from another derived class.



Hierarchical Inheritance: In this type of inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class.



Hybrid (Virtual) Inheritance: Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance. Below image shows the combination of hierarchical and multiple inheritances:



- Hybrid inheritance is also known as *Virtual Inheritance*.
- It is a combination of two or more inheritance.
- In hybrid inheritance, when derived class have multiple paths to a base class, a diamond problem occurs. It will result in duplicate inherited members of the base class.
- To avoid this problem easily, we use **Virtual Base Class**. In this case, derived classes should inherit base class by using Virtual Inheritance

Ambiguity in Multiple Inheritance

When Two base classes have functions with the same name, while a class derived from both base classes.

Virtual Function

1. **What is virtual function? (AU22) (SP22)**
   Ans: A virtual function is a member function in a base class that is declared with the keyword "*virtual*".
2. Why do we need virtual functions? / What are the reason to use virtual function? **(AU22) (SP22)**
   Ans:

   i. Runtime Polymorphism
   Virtual functions enable runtime polymorphism, which is the ability of the program to resolve function calls at runtime rather than at compile time. This allows for more flexible and dynamic behavior in programs.

   ii. Dynamic Binding
   Virtual functions ensure that the correct function implementation is called based on the actual object type, not the type of the pointer or reference. This dynamic binding or late binding is essential for implementing polymorphic behavior.

   iii. Code Reusability
   By using virtual functions, we can write code that works with objects of different classes in a uniform way. This allows you to write general-purpose functions that can operate on objects of various derived classes through base class pointers or references.

   iv. Extensibility
   Virtual functions allow new derived classes to be added with minimal changes to existing code. You can extend the functionality by adding new derived classes that override virtual functions without modifying the base class or other parts of the program.

   v. Abstract Interfaces
   Virtual functions can be used to create abstract base classes with pure virtual functions. These abstract base classes define interfaces that must be implemented by derived classes, ensuring a consistent interface across different implementations.

   vi. Separation of Interface and Implementation
   Virtual functions help separate the interface from the implementation. The base class provides the interface, while derived classes provide the specific implementations. This separation enhances modularity and makes the code easier to understand and maintain.

3. **When do we make a virtual function "*pure*"? what are the implication of making a function a pure virtual function? Explain with example. (AU23)**
   Ans:
   A pure virtual function is a virtual function that is declared by assigning 0 to it within the base class.
   When do we make a virtual function "*pure*":
   i. When we want to create an abstract base class that provides a common interface for all derived classes, but we don't want to provide a default implementation in the base class.
   ii. When we want to ensure that all derived classes must implement their own version of the function
   Implications of Making a Function Pure Virtual:
   Abstract Base Class: The class containing a pure virtual function becomes an abstract class and cannot be instantiated directly.
   Polymorphism: Pure virtual functions enable polymorphism, allowing you to work with objects of different derived classes through a common base class pointer or reference.
   Forced Implementation: Derived classes must override the pure virtual function to provide a concrete implementation. If they don't, they also become abstract classes.

4. **Explain the difference between early binding and late binding in C++. Discuss the pros and cons of them. (AU23) (AU22) (SP22) (SP23)**

Ans:

Early Binding: Early binding, also known as static binding or compile-time binding, occurs when the function call is resolved at compile time. If the compiler knows at the compile-time which function is called, it is called early binding.

```
#include<iostream>
using namespace std;
void display(int x){
cout << x;
}in
t main(){
display(5);
return 0;
}
```

Pros and Cons:

The main advantage of early binding is that very efficient. The main disadvantage is lack of flexibility.

Late Binding: Late binding, also known as dynamic binding or runtime binding, occurs when the function call is resolved at runtime. If a compiler does not know at compile-time which functions to call up
until the run-time, it is called late binding

```
#include<iostream>
using namespace std;
void display(int x){
cout << x;
}
int main(){
void (*ptrFunc)(int) = display; // define a function pointer type
ptrFunc(5);
return 0;
}
```

Pros and Cons:

The main advantage of late binding is flexibility at run time. The disadvantage is that there is more overhead associated with a function call.

5. **How virtual function are used to implement late binding? (AU23)**

Ans:

```
class Shape {
public:
virtual void draw() = 0; // pure virtual function
};
class Circle : public Shape {
public:
void draw() {
// draw circle
}
};
class Rectangle : public Shape {
public:
void draw(){
// draw rectangle
}
};
int main() {
Circle c;
Rectangle r;
Shape* ptr;
ptr = &c; // assign pointer to Circle object
ptr->draw(); // call draw() on Circle object
ptr = &r; // assign pointer to Rectangle object
ptr->draw(); // call draw() on Rectangle object
return 0;
}
```

6. **Write a program by using pure virtual function? (AU22)**

Ans:

```
#include<iostream>
using namespace std;
class B {
public:
virtual void s() = 0; // Pure Virtual Function};
class D:public B {
public:
void s() {
cout << "Virtual Function in Derived class\n";
}};
int main() {
B *b;
D dobj;
b = &dobj;
b->s();
}
```

7. **What do you mean by polymorphism in OOP? Give the examples of static and dynamic polymorphism. (AU22)**

Ans:

Example of static polymorphism:

```
#include <iostream>
using namespace std;
// Function to add two integers
int add(int a, int b) {
    return a + b;
}
// Function to add three integers
int add(int a, int b, int c) {
    return a + b + c;
}
int main() {
    int sum1 = add(10, 20);        // Calls the first add() function
    int sum2 = add(10, 20, 30);    // Calls the second add() function
    cout<< sum1 <<endl;
    cout<< sum2 <<endl;
    return 0;
}
```

Example of dynamic polymorphism:

```
class Shape {
public:
virtual void draw() = 0; // pure virtual function
};
class Circle : public Shape {
public:
void draw() {
// draw circle
}
};
class Rectangle : public Shape {
public:
void draw(){
// draw rectangle
}
};
int main() {
Circle c;
Rectangle r;
Shape* ptr;
ptr = &c; // assign pointer to Circle object
ptr->draw(); // call draw() on Circle object
ptr = &r; // assign pointer to Rectangle object
ptr->draw(); // call draw() on Rectangle object
return 0;
}
```

Template

Templates in C++ Programming

Templates in C++ programming allows function or class to work on more than one data type at once without writing different codes for different data types.

a. Templates are often used in larger programs for the purpose of code re-usability and flexibility of program.

b. The main advantage of using a template is the reuse of same algorithm for various data types, hence saving time from writing similar codes.

c. The concept of templates can be used in two different ways:

• Function Templates Or Generic functions

• Class Templates Or Generic classes

Generic Function:

ii. A generic function defines a general set of operations that will be applied to various types of data.

ii. A generic function that represents several functions performing same task but on different data types.

iii. A generic function is created using the keyword **template**.

Why use Function Templates

• Templates are instantiated at compile-time with the source code.

• Templates are used less code than overloaded C++ functions.

• Templates are type safe.

• Templates allow user-defined specialization.

• Templates allow non-type parameters

Generic Classes:

A generic class, also known as a template class, is a class that can be instantiated with different types.

→ Like function templates, we can also create class templates for generic class operations.

→ Sometimes, we need a class implementation that is same for all classes, only the data types used are different.

→ Normally, we would need to create a different class for each data type OR create different member variables and functions within a single class.

→ The general form of a generic class declaration is shown here:

```
template <class T > class class_name
{};
```

→ To create a class template object, we need to define the data type inside a < >.

Exception Handling

1.What is an exception?

Ans:

Exceptions are run time errors which occur at **runtime** like low disk space, division by zero, access to array outside its bounds. Exception Handling in C++ is a process to handle runtime errors or exceptions.

2.What is the advantage of exception handling?

Ans:

The benefits of exception handling are as follows,

(a) Exception handling can control run tune errors that occur in the program.

(b) It can avoid abnormal termination of the program and also shows the behavior of program to users.

(c) It can provide a facility to handle exceptions, throws message regarding exception and completes the execution of program by catching the exception.

(d) It can separate the error handling code and normal code by using try-catch block.

(e) It can produce the normal execution flow for a program.

3.Show the general form of *try*, *catch* and *throw.* Describe their operation?

Ans:

throw*:** A program throws an exception when a problem shows up. This is done using a ***throw keyword.

***catch*:** A program catches an exception with an exception handler at the place in a program where you want

to handle the problem. The ***catch*** keyword indicates the catching of an exception.

try: A ***try*** block identifies a block of code for which particular exceptions will be activated. It's followed by one or more catch blocks.

If an exception occurs within the **try** block, it is thrown (using **throw**). The exception is caught, using **catch** in the catch block, and processed.

STL

1.What is STL?

Ans: The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc,

2.Define a container, an iterator and algorithm as the relate to the STL?

Ans:

Container:

The container can be described as objects that are used to store the collection of data. The STL provides a range of containers, such as vector, list, map, set, and stack.

Iterator:

Iterators are used to access data in the containers, and it helps in traversing through elements of containers using its functions.

begin(): This function points the iterator to the first element of the container.

end(): This function points the iterator to the last element of the container.

Algorithm:

The algorithm defines a collection of functions specially designed to be used on a range of elements. Algorithms can be defined as functions applied to the containers and provide operation for the content of the container. for example: sort (), swap (), min (), max () ,find ( ), and binary, search() etc.

3.Operation of STL Vector?

Ans:

```
vector<int>::iterator it;
it = arr.begin();
```

Remove Element

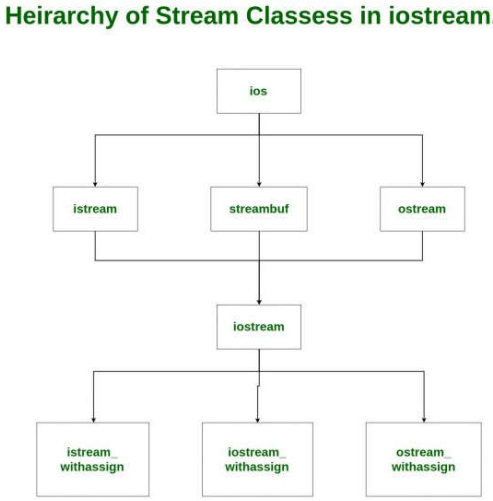
```
arrayname.erase(it+pos);          arrayname.popback();//>>last element remome
```

Insert Element

```
arr.insert(it,value);
```

I/O File

1. **What is a *stream*?**
   Ans: In C++, a stream is an abstraction that represents a sequence of characters that can be read from or written to. Streams provide a standardized way to perform input and output (I/O) operations
2. **Write a classes hierarchy for console I/O operation**
   Ans:



ios class is topmost class in the stream classes hierarchy. It is the base class for istream, ostream, and streambuf class.
istream and ostream Serves the base classes for iostream class. The class istream is used for input and ostream for the output.

Class ios is indirectly inherited to iostream class using istream and ostream. To avoid the duplicity of data and member functions of ios class, it is declared as virtual base class when inheriting in istream and ostream as get, getline, read, ignore, putback etc..

3. **What is *manipulator*?**
   Ans:
   A manipulator is a special function or object that is used to modify the behavior of input/output streams.
4. **Formulate the difference between manipulator and ios member funtions?**
   Ans:

manipulator	**ios member funtions**
manipulators needs <iomanip>	ios function needs <iostream>
we can create our own manipulators.	we can not create own ios functions
manipulators are non-member functions.	ios functions are member functions
manipulators does not returns value	ios functions returns value
Examples: setw, setprecision, fixed, hex, endl, etc.	Examples:width, precision, flags,clear, seekg, flush, etc

Formatting using the ios members:
The stream has the format flags that control the way of formatting. it means Using this setf function, we can set the flags, which allow us to display a value in a particular format.
Few standard ios class functions are:
width(): The width method is used to set the required field width. The output will be displayed in the given width
precision(): The precision method is used to set the number of the decimalpoint to a float value
fill(): The fill method is used to set a character to fill in the blank space of afield
setf(): The setf method is used to set various flags for formatting output
unsetf(): The unsetf method is used To remove the flag setting

Formatting using Manipulators:

The second way to format parameters of a stream is through the use of special functions called manipulators that can be included in an I/O expression.The standard manipulators are shown below:

- **boolalpha:** The boolalpha manipulator of stream manipulators in C++ is used to turn on bool alpha flag
- **dec:** The dec manipulator of stream manipulators in C++ is used to turn on the dec flag
- **endl:** The endl manipulator of stream manipulators in C++ is used to Output a newline character.
- **and:** The and manipulator of stream manipulators in C++ is used to Flush the stream
- **ends:** The ends manipulator of stream manipulators in C++ is used to Output a null
- **fixed:** The fixed manipulator of stream manipulators in C++ is used to Turns on the fixed flag
- **hex:** The hex manipulator of stream manipulators in C++ is used to Turns on hex flag
- **internal**: The internal manipulator of stream manipulators in C++ is used to Turns on internal flag
- **left**: The left manipulator of stream manipulators in C++ is used to Turns on the left flag
- **noboolalpha:** The noboolalpha manipulator of stream manipulators in C++ is used to Turns off bool alpha flag
- **noshowbase:** The noshowbase manipulator of stream manipulators in C++ is used to Turns off showcase flag
- **noshowpoint:** The noshowpoint manipulator of stream manipulators in C++ is used to Turns off show point flag
- **noshowpos**: The noshowpos manipulator of stream manipulators in C++ is used to Turns off showpos flag
- **noskipws:** The noskipws manipulator of stream manipulators in C++ is used to Turns off skipws flag
- **nouppercase:** The nouppercase manipulator of stream manipulators in C++ is used to Turns off the uppercase flag